

When can a vision language model stop looking at the image?

Measuring the depth at which visual tokens stop mattering, and what token pruning does with that

P. S. Kedar

Supervised by Prof. Divya Saxena | Mentored by Aditya Sharma, PhD Scholar | IIT Jodhpur, September 2026

1. What we were chasing

Token pruning throws away visual tokens while a model runs, to save compute. Balanced Token Pruning does it in three stages. We reproduced its published numbers in earlier phases and they held up, except on one thing: Qwen2.5-VL-7B lost almost all its ability to read text inside an image. TextVQA fell from 86.2 to 23.3. DocVQA fell from 94.7 to 19.2. AI2D, which has text but rarely needs it read word by word, barely moved.

That gap is what Phase 3 set out to explain. The answer took four failed experiments to reach, and it turned out to sit in a place none of us were looking.

What we found

There is a depth past which deleting every visual token costs a model almost nothing, and before which it is ruinous. The change is abrupt. We measured it at 74.8 per cent of the way through InternVL2-2B and 80.3 per cent through Qwen2.5-VL-7B.

BTP's released code deletes at a point that leaves Qwen 22 layers of image access. We measure the threshold at 22.5. One layer either side is worth about 47.7 percentage points on TextVQA.

Deletion is not the whole story. On ChartQA the ordinary pruning costs twice what the deletion does, so we keep the two apart throughout.

None of this shows up on the benchmarks the paper reports. With the fault in place the code still scores 96.1 per cent of baseline on five of them.

We measured the thresholds on TextVQA only, on two models, and our InternVL pruning is not a complete reproduction of BTP. Section 6 says where that bites.

2. Four things that were not the cause

We listed what could explain the collapse and tried to kill each one.

What we thought	What we did	What came back
The selector picks badly	Swapped it for diversity only, attention only, and random, at the same budget	17.3 to 19.4, standard errors 2.6 to 2.7. Random did as well as BTP
Text patches look alike, so a diversity sampler thins them	Measured feature similarity at layer 4	Text-text 0.379, background-background 0.438. Text is the more varied region, so our premise was wrong
Too many tokens are removed	Swept retention from 100 down to 12.5 per cent	Keeping 90 per cent already gave the full drop, 86.2 to 23.3. Keeping 12.5 gave 23.3
The right tokens are not being kept	Used ground-truth OCR to force every text token to survive	0.197 with the oracle, 0.197 without

Table 1. Four negatives, in order. All four landed on the same floor.

Four unrelated interventions, one number. That should have been the clue and it was not, until we added a control at 100 per cent retention, where nothing is pruned at all. It scored 0.178. Keeping every token changed nothing, so whatever was destroying the image was happening after the pruning stages, not during them.

3. The line, and what happens when you remove it

We read through the released Qwen file for anything that touches the hidden-state sequence. One branch does this:

```
elif layer_index == self.end_layer:      # self.end_layer = 23
    remain_sys_index = torch.tensor(list(range(self.img_start_idx)))
    remain_text_index = torch.tensor(list(range(img_end, total_length)))

    all_remained_index = torch.cat((remain_sys_index, remain_text_index))
    hidden_states = hidden_states[:, all_remained_index, :]
```

System tokens and text tokens go into the concatenation. Image tokens do not. There is no `remain_img_index`, so everything that survived three stages of careful pruning is thrown away here regardless. Directly above it sits a commented-out version that would have kept some by attention score.

Reading code is not evidence. We changed the one condition so the branch can never fire, left the 12.5 per cent pruning exactly as it was, and ran it again.

Arm	TextVQA	AI2D
No pruning	0.8565	0.8750
BTP as released	0.1725	0.8150
Deletion disabled	0.7860	0.8100

Table 2. Job 416735, 200 samples. The 0.8565 baseline differs from the 86.2 used elsewhere because this ran on a separate 200 sample subset. Code: `NeurIPS2025-Balanced-Token-Pruning` at commit 9682db0, file `qwen-2.5-vl/modeling_qwen2_5_vl.py`, constants at lines 1140 to 1144, branch at line 1385, read unmodified.

One condition, and reading goes from 0.1725 to 0.7860. AI2D does not move, which matters, because it means we did not simply find a lever that lifts every score.

The paper is worth checking here. Appendix 7.3 specifies complete deletion for the LLaVA family. For Qwen2.5-VL it asks for the opposite: keep 12.5 per cent in the final stage, in the authors' words, to preserve model performance. The released code we looked at applies the LLaVA branch to Qwen anyway. That is a discrepancy in the released implementation. It says nothing about what the authors intended, and nothing at all about Qwen2.5-VL as a model.

4. Is layer 23 special, or just early?

Finding a bad constant is less interesting than finding out what makes it bad. So we moved the deletion across the whole decoder with the graded pruning switched off, changing nothing else, and did the same on InternVL2-2B. That model shares almost nothing with Qwen: a 24 layer InternLM2 backbone, ordinary 1D RoPE instead of M-RoPE, 448 pixel tiling instead of a merged patch grid.

A note on counting

We report the number of layers that still had the image, counting from the first: deleting before layer 12 of 28 means twelve had it and sixteen did not. That is not the number a method writes in its code. BTP counts from one and deletes before the named layer, so its end_layer of 23 leaves twenty-two layers with access. We count from zero. An earlier draft of this report conflated the two.

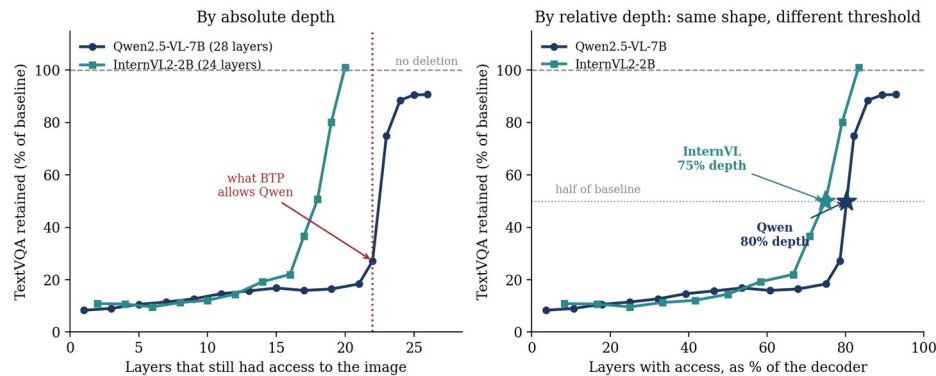


Figure 1. Left, by layers with access. Right, by relative depth. Both models sit on a floor, then climb sharply.

We take the threshold to be where accuracy crosses half the unpruned baseline, interpolated between the two measured layers either side. A different cutoff, say 90 per cent, moves both numbers, so treat these as a comparison between the models rather than absolutes.

Model	Decoder layers	Threshold	Relative depth
InternVL2-2B	24	18.0	74.8 %
Qwen2.5-VL-7B	28	22.5	80.3 %

Table 3. Layers that still need the image.

Two things follow. The shape is the same in both models, which makes this look like a property of the architecture rather than a quirk of one. And the depths differ, which means you cannot guess the number from the layer count.

BTP's constant leaves Qwen 22 layers. We measure 22.5. At 22 layers the model keeps 27.2 per cent of baseline; at 23 it keeps 74.8. So the released value sits just under the edge, and one layer is worth roughly 47.7 percentage points.

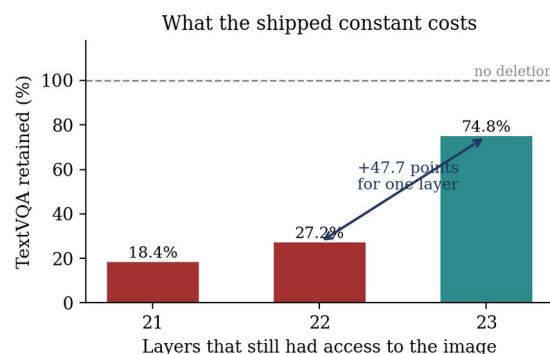


Figure 2. Either side of the threshold. The 47.7 comes from the unrounded scores; the rounded percentages shown differ by 47.6.

A caveat about what this is. We measure what happens when the tokens are removed. We do not observe the model using them, so we describe sensitivity, not mechanism. Calling it the point where the model stops looking would be reading more into the curve than it contains.

5. Checking the measurement with a different instrument

Both sweeps edit model source, separately for each architecture, which nobody else can reuse. So we rebuilt the measurement as a forward hook that removes the tokens during the pass and touches no source at all, then ran it on Qwen where we already knew the answer.

Layers with access	Source patch	Forward hook
13	15.7 %	17.0 %
22	27.2 %	27.5 %
23	74.8 %	75.8 %
24	88.5 %	86.3 %

Table 4. Job 423959, different sample sets, so compare the threshold and not the absolute scores.

Same shape, a couple of points apart. That is reassuring about the measurement rather than about any particular way of making it, and we have only done the comparison on one model. The tool needs no training and no calibration set, and a coarse sweep plus bisection finds the threshold in about seven runs.

One difference at the top of the curve we did not expect. InternVL comes all the way back, 101 per cent of baseline with 20 of its 24 layers. Qwen stops at 90.7 per cent even with 26 of 28. Something in Qwen still wants the image very late, for some fraction of examples. We cannot say which, or how many, from this experiment.

6. Two costs, not one

With the mechanism understood we ran a corrected arm across the benchmarks. It still prunes to 12.5 per cent; it just does not delete what is left. That gives two gaps worth separating. Baseline to corrected is what pruning costs. Corrected to released is what the deletion costs on top.

Benchmark	Baseline	Released	Corrected	Pruning cost	Deletion cost
TextVQA	86.2	23.3	80.5	5.7	57.2
DocVQA (ANLS)	94.7	19.2	76.8	17.9	57.6
ChartQA	76.8	29.0	45.4	31.4	16.4
AI2D (control)	86.4	79.6	79.2	7.2	0.4
GQA	60.9	55.9	58.8	2.1	2.9
POPE (accuracy)	87.6	86.2	86.3	1.3	0.1
MME (perception)	1674.5	1658.7	1651.2	23.3	7.5
MMBench-EN	83.7	79.3	79.0	4.7	0.3
ScienceQA-Image	88.1	85.1	85.0	3.1	0.1

Table 5. Qwen2.5-VL-7B. MME is scored out of roughly 2000 rather than 100, so its last two figures are 1.4 and 0.4 per cent of its own baseline. Where a deletion cost would be negative, meaning the released arm scored a shade higher, we give the magnitude.

ChartQA is the row that stops you. There the deletion costs 16.4 points and the pruning costs 31.4, so the fault is not the main problem on that benchmark. We had been writing the deletion as the dominant cause everywhere. It is not.

6.1 What the standard suite reports

The paper reports six benchmarks. We ran five of them: GQA, MME, MMBench, POPE and ScienceQA. MM-Vet is out because a language model judges it, which adds a second source of variance to a comparison meant to isolate one line of code.

Averaged across those five, the released arm keeps 96.1 per cent of baseline and the corrected arm 96.9. A gap of 0.8 points. On the three reading tasks the same two arms give 28.4 and 77.9 per cent. The suite the method was validated on cannot see the difference.

6.2 The same blind spot without any fault

We then pruned InternVL2-2B to 12.5 per cent with no deletion at all, on an implementation where we found nothing of the kind.

Benchmark	Baseline	Pruned to 12.5 %	Retained
TextVQA	0.716	0.468	65.4 %
DocVQA (ANLS)	0.835	0.303	36.3 %
ChartQA	0.586	0.398	67.9 %
AI2D (control)	0.746	0.728	97.6 %
GQA	0.592	0.546	92.2 %
POPE (accuracy)	0.888	0.848	95.5 %
MMBench-EN	83.4	81.0	97.1 %

Table 6. InternVL2-2B, pruning only.

The three reading tasks average 56.5 per cent of baseline. The three from the paper's suite that we ran here, GQA, POPE and MMBench, average 94.9. AI2D is at 97.6 but it is our control, not part of that suite. In points lost: 43.5 against 5.1, about 8.6 to one. No bug anywhere, correct pruning, and the standard benchmarks still report 95 per cent while the model has lost half its reading.

This arm is not a full reproduction of BTP. Our selector has the diversity component but not the attention one, because InternLM2's FlashAttention-2 path does not expose attention scores, and we scaled the pruning layers across from Qwen rather than calibrating them. In one probe on one image, attention picks landed on text 36.0 per cent of the time against a 28.8 per cent share, so attention does seem to favour text. One image cannot tell us by how much, which makes these numbers a likely upper bound on what BTP itself would cost.

7. Where this leaves us

Three claims, at three levels, and they stand or fall separately.

- **Implementation.** The released BTP code we read deletes all remaining image tokens just below the threshold we measured for Qwen2.5-VL, while the paper's appendix asks for retention on that model.
- **Model.** Visual token access has a measurable depth threshold in both models we tested, and the two differ.
- **Evaluation.** The suite we evaluated did not show the damage to text reading, with the fault or without it.

The third is the one that travels furthest, and it does not need the first to be true. On Qwen there is a fault the benchmarks cannot see. On InternVL there is no fault and the benchmarks still cannot see what pruning costs.

7.1 What we would not claim

- Two architectures is two architectures. The shape matched; that is not a law.
- The thresholds come from TextVQA. Given how ChartQA behaved in Table 5, we would not assume it sits in the same place there.
- The InternVL implementation is ours, with the selector half built and the layers transplanted rather than calibrated.
- The OCR oracle ran at one budget. An unexplained slowdown killed the sweep and we left it.
- Everything here uses 500 sample subsets, matched across arms, so the comparisons hold but the absolute numbers are not the published full-dataset ones.
- **Two of our own claims are withdrawn.** Phase 2 blamed the collapse on text having no redundancy; our own similarity measurement says the opposite, 0.379 against 0.438. And an early result where text coverage seemed to predict correct answers disappeared at a larger sample. An earlier draft of this report also concluded the effect was Qwen-specific, on one measurement in InternVL, and the sweep showed that was wrong.

7.2 What to do about it

Measure the depth instead of picking it. The sweep is cheap, the tool is written and validated on one model, and it needs no training or calibration set. Seven runs gets you the number.

And put one reading benchmark in the evaluation. Not because reading matters more than reasoning, but because it breaks first when visual evidence goes missing.

Next for us: thresholds on DocVQA and ChartQA, ChartQA first since pruning already costs more there than deletion does, then a third architecture, then the manuscript.

7.3 Where to check any of this

Run-by-run record	phase3-mechanism/RESULTS_LOG.md, observed output marked VERIFIED, untested statements HYPOTHESIS
Scripts and jobs	phase3-mechanism/, job scripts e9 to e13 and their submit chains
Threshold tool	phase3-mechanism/measure_visual_depth.py, runs on any Hugging Face VLM
Figures and this file	report/make_depth_figures.py and report/build_report.js, so every number comes out of a script
Repository	milab-iitj-dev/btp-reproduction, not public yet, local commit 139af1c
Jobs	411681, 412401, 414662, 416734-5, 416896-904, 417968, 418974-419012, 421205-421237
The paper	Balanced Token Pruning, NeurIPS 2025, arXiv:2505.22038. Code read at commit 9682db0.